Machine Learning

Lecture 16: Dimensionality Reduction

Spring 2025

Instructor: Xiaodong Gu



Unsupervised Learning Again

Any other patterns beyond data clusters?





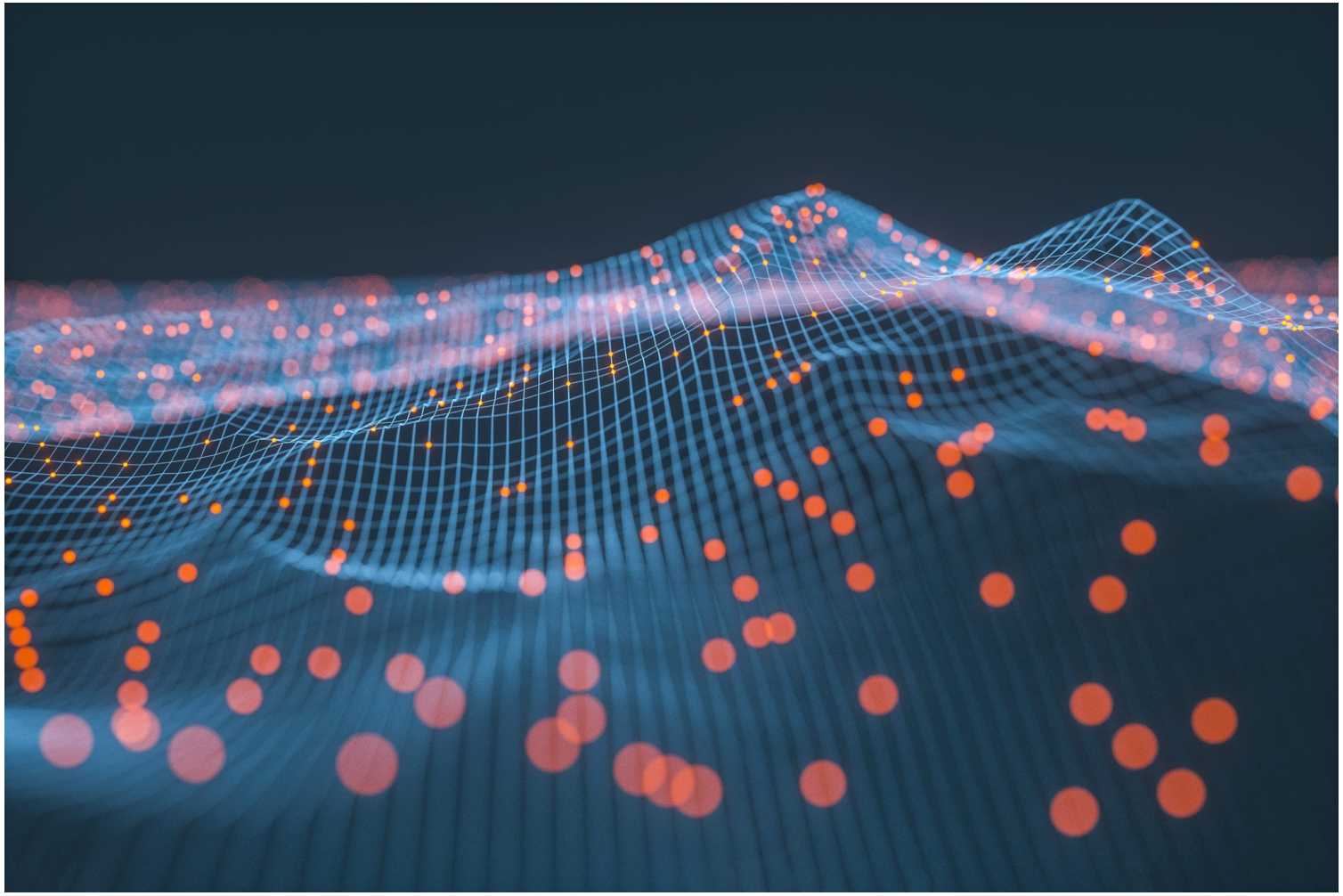


VISITORS	CONTACTS	LEADS	MQLS	BI_USER	CUSTOMERS	EXPECTED_ARR
14K	2.0K	1.650K	76.0	78.0	2	2.6M
↑37.2%	↑19.1%	↑21.0%	↑35.7%	↑69.6%	↑ N/A	↑2.3%

Medium | Source | Group Channel

GROUP CHANNEL	SOURCE	VISITORS	% Δ	LEADS	% Δ	LEADS TO MQLS CR	% Δ	MQLs	% Δ
Other	webinar	5.996	31.1% ↑	152	76.7% ↑	8%	69.7% ↑	12	200.0% ↑
Organic Traffic	yandex	3.424	18.8% ↑	102	18.6% ↑	2%	-15.7% ↓	2	0.0%
Referral Traffic	leadmagnet	918	9.08% ↑	1.172	18.1% ↑	4%	0.0% ↑	52	18.2% ↑
Brand Traffic	leadmagnet	722	65.6% ↑	2	58.3% ↑	5%	0.0% ↑	2	0.0%
Offline	Event	346	-3.4% ↓	2	-66.7% ↓	0%	0.0% ↑	0	0.0%
Referral Traffic	evernote.com	202	260.7% ↑	6	-88.9% ↓	0%	0.0% ↑	0	0.0%

Big Data, are they all useful?



Today

物以类聚

Clustering
(Learning data similarities)

- K-means
- Gaussion Mixture

無中生有

Generative
(density estimation)

- GAN
- VAE
- Diffusion



Dimensional Reduction
(data compression)

- Principal component analysis
- Auto-encoder

The Curse of Dimensionality

"As the number of dimensions grows, the amount of data we need to generalize accurately grows exponentially."



Charles Isbell, Georgia Tech

Example:

Given $(x_1,\ldots,x_N)$ with binary attributes, estimate $p(x_1,\ldots,x_N)$

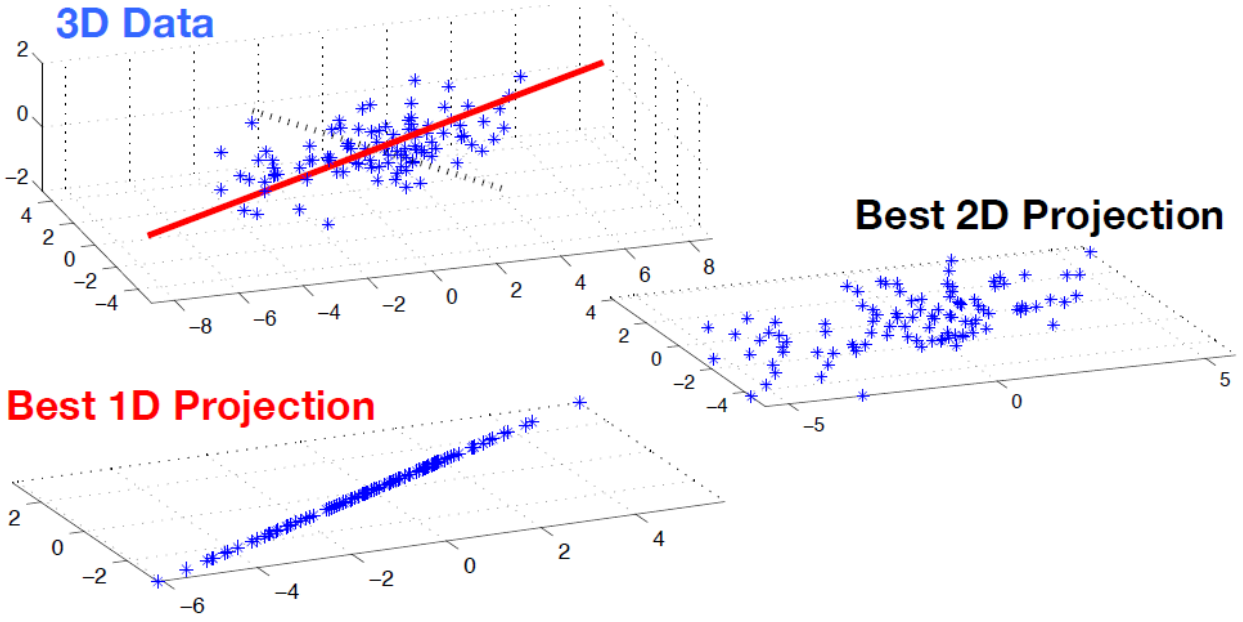
X_1: gender	X_2: major	X_3: age	…	$P(x_1,x_2,x_3)$
Male	CS	<25	…	0.9
Female	Finance	>25	…	0.1
…	…	…	…	…
Male	CS	<25	…	0.1

Q: How many items are needed in this table?

Dimensionality Reduction

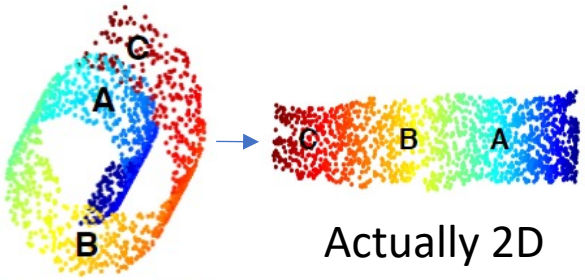
Reduce the number of dimensions (attributes) of the data while preserving the intrinsic characteristics of the data

Try to identify some "hidden" aspects that determines the characteristics of the data



Why Reduce Dimensionality?

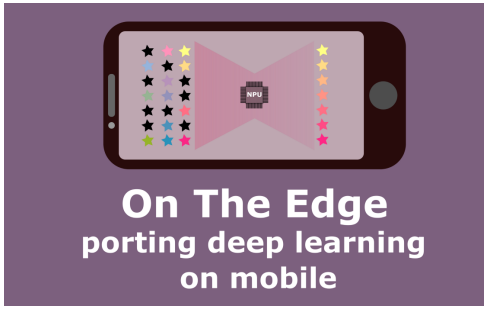
Data may intrinsically live in a lower-dim space



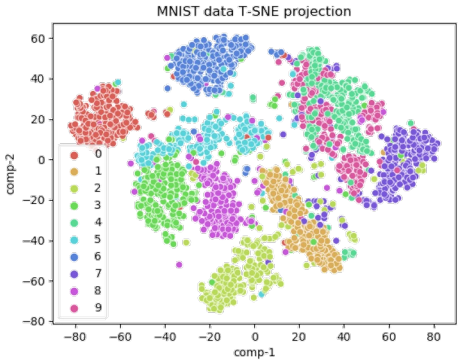
Too many features and too few data

X_1:	X_2:	X_3:	…	$P(x_1,x_2, x_3)$
M	CS	<25	…	0.9
F	Fin	>25	…	0.1
…	…	…	…	…
M	CS	<25	…	0.1

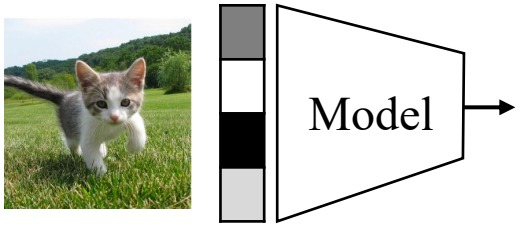
Lower computational expense (memory, time)



Want to visualize data in a lower-dim space



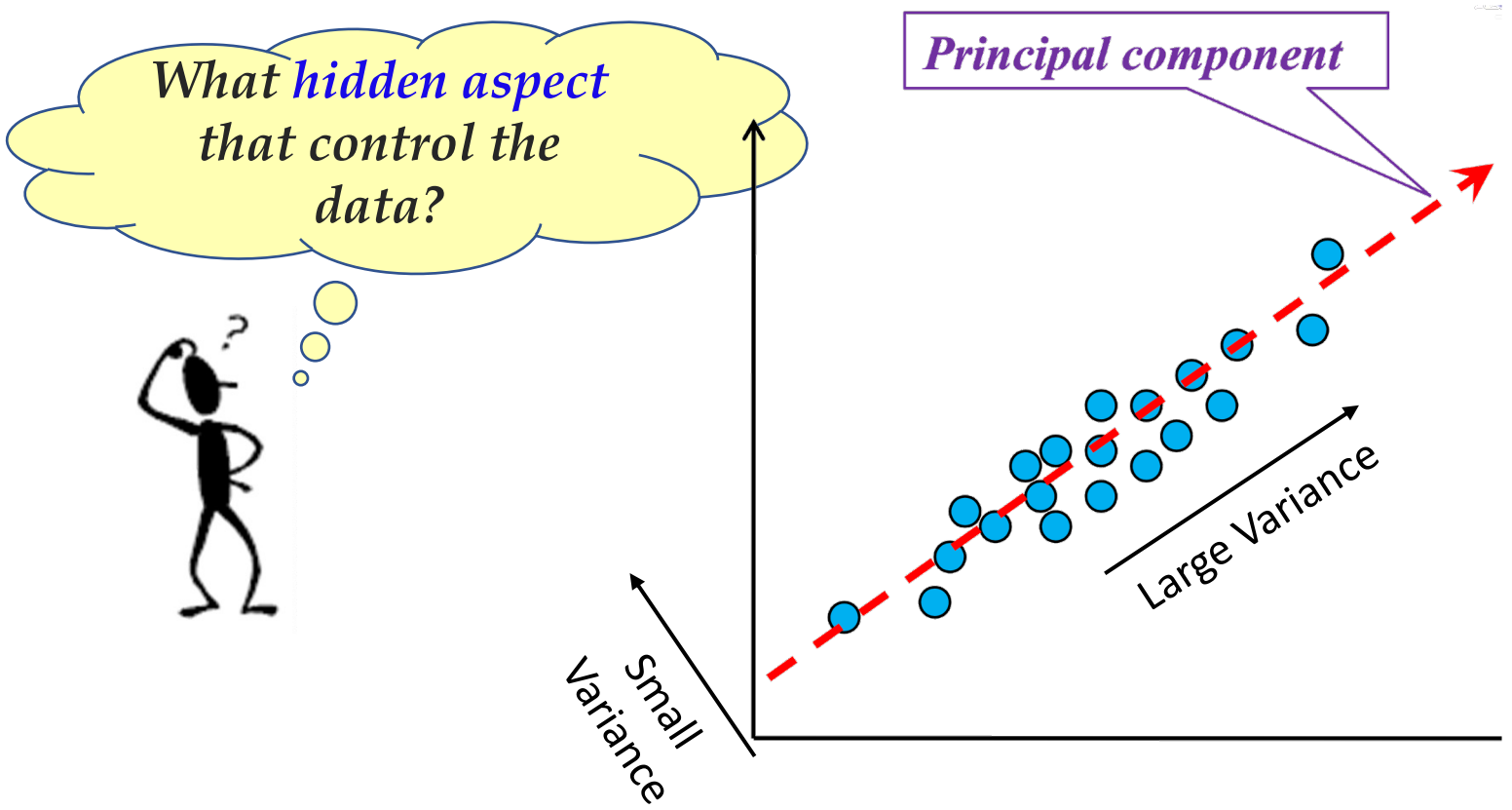
Want to use data of different dimension



Overview of Methods

- Linear Method
  - **Principal component analysis** (PCA)
  - Linear discriminate analysis (LDA)
  - Factor analysis
  - ...
- Non-linear Method
  - KPCA, KLDA
  - Multidimensional scaling (MDS)
  - t-SNE
  - **Autoencoder**
  - ...

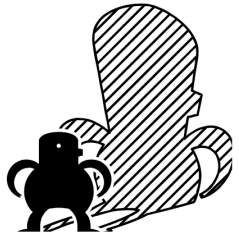
Principal Component Analysis (PCA)



Principal component = the spreading tendency

= dimensionality with the maximum variance

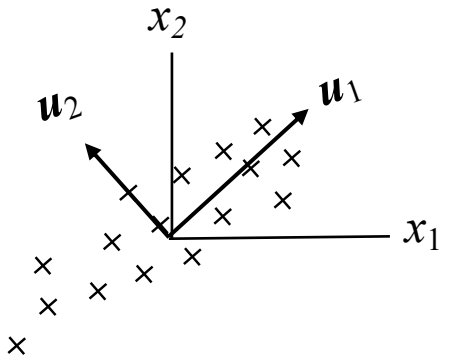
Problem Formulation



Can simply subtract each x_i by the sample mean.

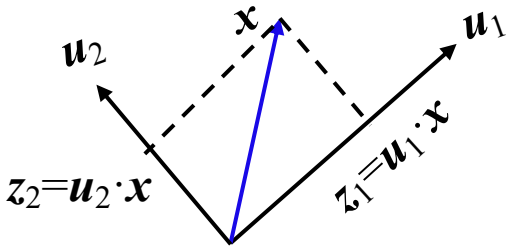
- Data representations

$$\boldsymbol{X} = [x_1, \ldots, x_N],\ x_i \in \mathbb{R}^d, \text{ where } \sum_{i=1}^{N} x_i = 0$$



- Projection of a data point

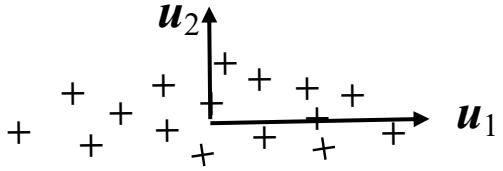
$$z_j = \boldsymbol{u}_j^T x \qquad \boldsymbol{Z} = \boldsymbol{U}^T \boldsymbol{X}$$



- Variance in any projected dimension

$$var(\boldsymbol{u}^T\boldsymbol{X}) = \tfrac{1}{N} (\boldsymbol{u}^T\boldsymbol{X})(\boldsymbol{u}^T\boldsymbol{X})^T$$
$$= \tfrac{1}{N} \boldsymbol{u}^T\boldsymbol{X}\boldsymbol{X}^T\boldsymbol{u}$$
$$= \boldsymbol{u}^T\boldsymbol{S}\boldsymbol{u}$$

$S = \mathrm{cov}(X) = \tfrac{1}{N} XX^T$



PCA = Solving an Optimization Problem

- In summary, PCA aims to solve a constrained optimization problem:

$$\max_{\mathbf{u}} \ \boldsymbol{u}^{\mathrm{T}}\boldsymbol{S}\boldsymbol{u}$$

$$\text{s.t.} \ \ \boldsymbol{u}^{\mathrm{T}}\boldsymbol{u} = \mathbf{1}$$

- This is a quadratic programming (QP) problem, which is one type of convex optimization problem.
- S is positive **semi-definite**.

$$\begin{aligned} x^T S x &= x^T E[(X-\mu)^T(X-\mu)]x \\ &= E[x^T(X-\mu)^T(X-\mu)x] \\ &= E[((X-\mu)x)^T((X-\mu)x)] \\ &= E[\|(X-\mu)x\|^2] \\ &= \|(X-\mu)x\|^2 \\ &\geqslant 0 \end{aligned}$$

Solving PCA

- Lagrangian: $\mathcal{L} = \boldsymbol{u}^{\mathrm{T}}\boldsymbol{S}\boldsymbol{u} - \lambda(\boldsymbol{u}^{\mathrm{T}}\boldsymbol{u} - 1)$

$$\nabla_{\boldsymbol{u}}\mathcal{L} = 0 \Rightarrow \boldsymbol{S}\boldsymbol{u} = \lambda\boldsymbol{u}$$

- Solving PCA subjects to **eigen decomposition**:

  $\boldsymbol{u}$: eigen vector, λ: eigen value

- Since $\boldsymbol{S}$ is positive semi-definite, we have

$$\boldsymbol{S} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{U}^{\mathrm{T}} = [\boldsymbol{u}_1, \ldots, \boldsymbol{u}_{\mathrm{d}}]\mathrm{diag}(\lambda_1, \lambda_2, \ldots, \lambda_{\mathrm{d}})[\boldsymbol{u}_1, \ldots, \boldsymbol{u}_{\mathrm{d}}]^{\mathrm{T}}$$

The Idea

- Find k orthogonal principal components

  correspond to the k eigen vectors with k **largest** eigen values

$$S = U\Sigma U^T \approx U_{1:k}\Sigma_{1:k}U^T_{1:k}$$

- Project each input vector x into this subspace

$$z_j = u_j^T x \qquad z = U_{1:k}^T x$$

Intuitions:

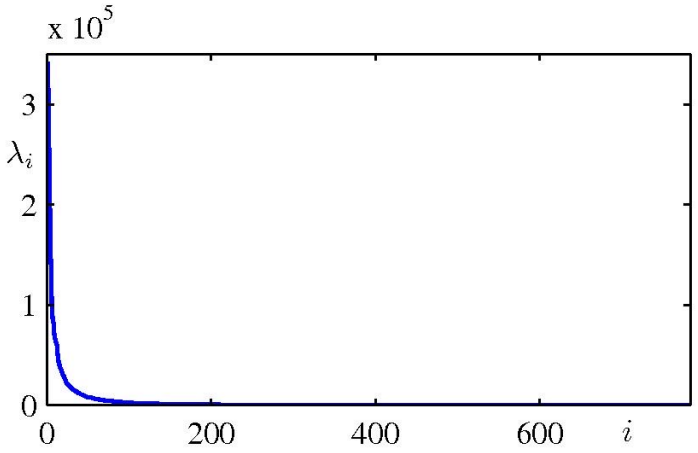
– *Eigen value is the variance of the data projected to the corresponding eigen vector*

– *Eigen vectors are essentially orthogonal*

Determine "k"

- **What to discard**: the eigen vectors with small eigen values
- Limit the loss of information within an acceptable interval (usually 5%)

$$\text{info loss} = \frac{\lambda_{k+1} + \ldots + \lambda_d}{\lambda_1 + \lambda_2 + \ldots + \lambda_d}$$



Variance preserved at *i*-th eigenvalue

The Algorithm

1. Shifting data set to have zeros mean

2. Compute the covariance matrix S

3. Conduct eigen decomposition on S, and rank the eigen vectors according to their eigen values

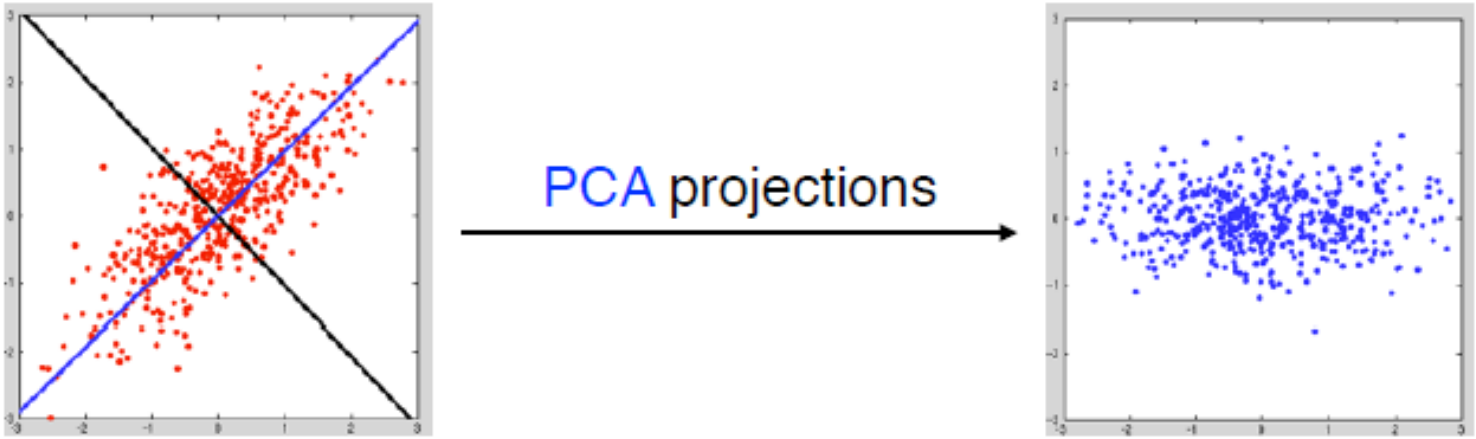
4. Determine the number of dimensions *k* of the new space

5. Select the first *k* eigen vectors with d largest eigen values as the basis of the new space

The Algorithm

Matlab Code

```
function [E, U, lambda] = PCA(X, K)
mu = mean(X);                    % Data mean
N = size(X,1);                   % Number of data
X = X - ones(N,1)*mu;            % Center the data
covX = X'*X;                     % Data covaraince
[U,lambda] = eigs(covX, K);      % Compute top K eigenvalues
E = X*U;                         % Compute embeddings
```



Slide credit: Subhransu Maji

Applications of PCA



- Data visualization
- Preprocessing
- Modeling – prior for new data
- Compression

Application: Face Recognition

- The space of face images are extremely high-dimensional
  - 24x24 image = 576 dimensions
  - slow and lots of storage



But very few 576-dimensional vectors are valid face images

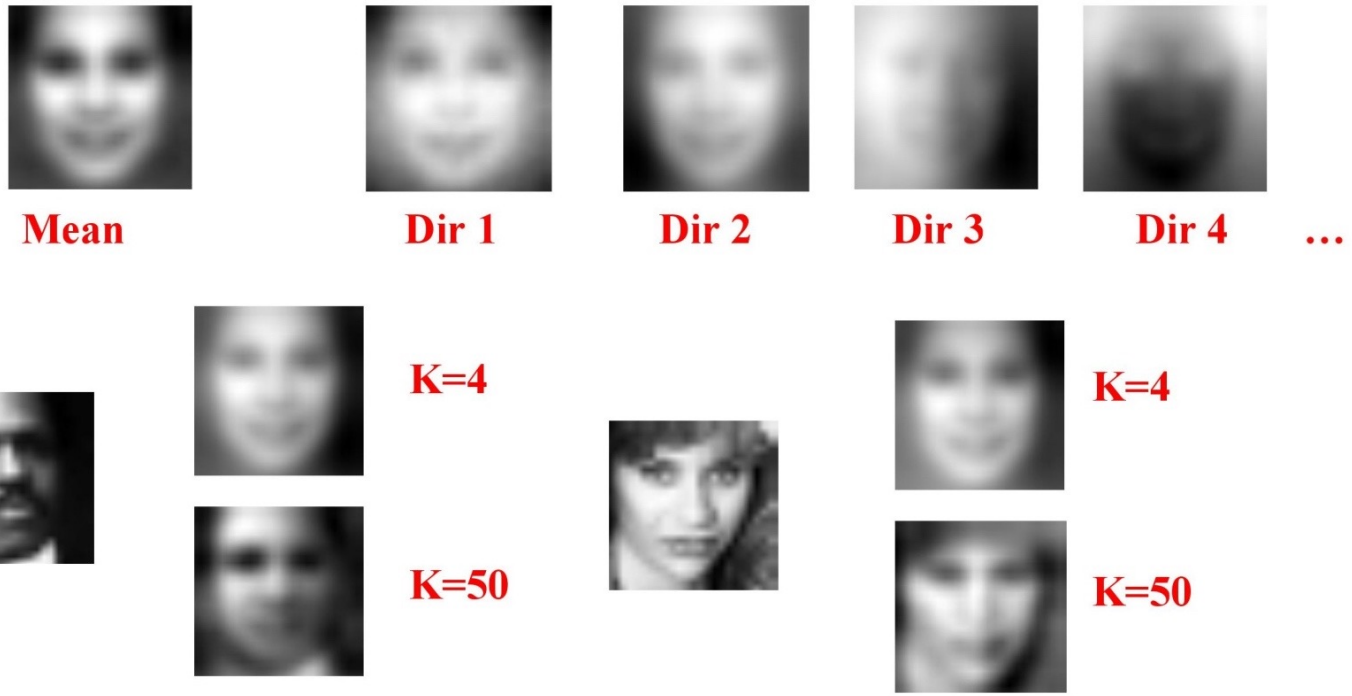
How to effectively model the subspace of face images?

Adapted from Derek Hoiem

"Eigen-Faces"

Example: Viola Jones data set

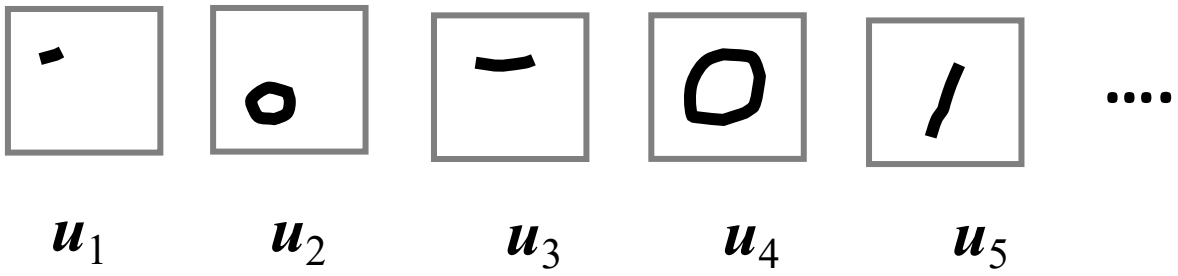
- 24×24 images of faces = 576 dimensional measurements
- Take the first-K PCA components



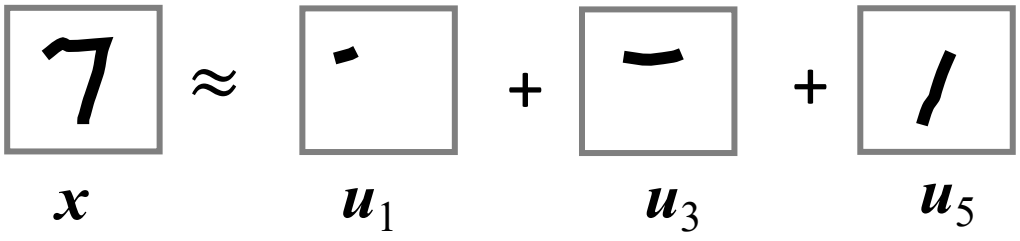
Slide credit: Alexander Ihler

PCA – Another Perspective

Basic components of handwriting digits:



u_1 u_2 u_3 u_4 u_5



$x \approx u_1 + u_3 + u_5 \Rightarrow \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 1 \\ \vdots \end{bmatrix}$

$$x \approx z_1 u_1 + z_2 u_2 + \ldots + z_k u_k + \bar{x}$$

Image pixels

components

sample mean

$$\begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_k \end{bmatrix}$$

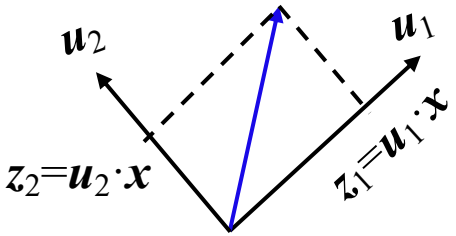
Compressed representation

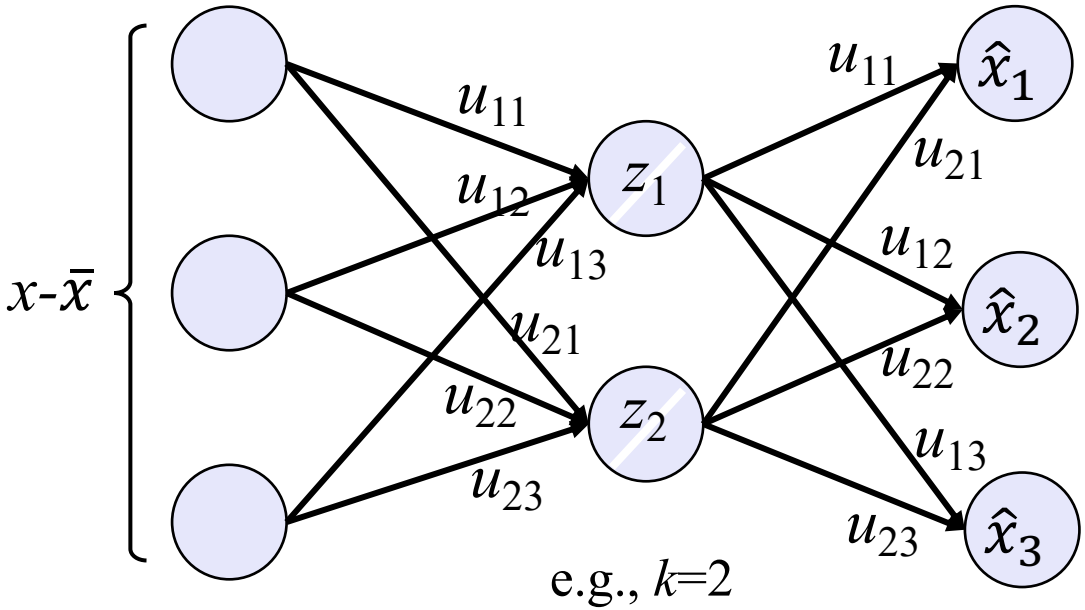
PCA = Neural Network

$\hat{x} = \sum_{j=1}^{K} z_j \boldsymbol{u}_j \iff x - \bar{x}$

To minimize reconstruction error:

$z_j = (x - \bar{x}) \cdot \mathrm{u}_j$





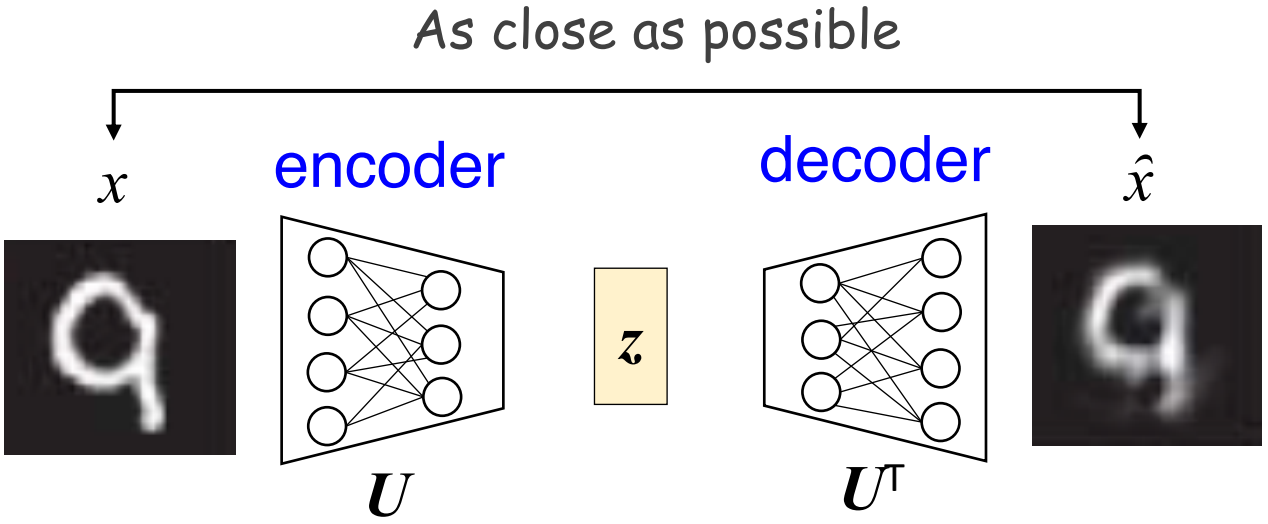
$\iff$ $x - \bar{x}$

minimize error

PCA = a neural network with one hidden layer (linear activation)

PCA = Auto-encoder

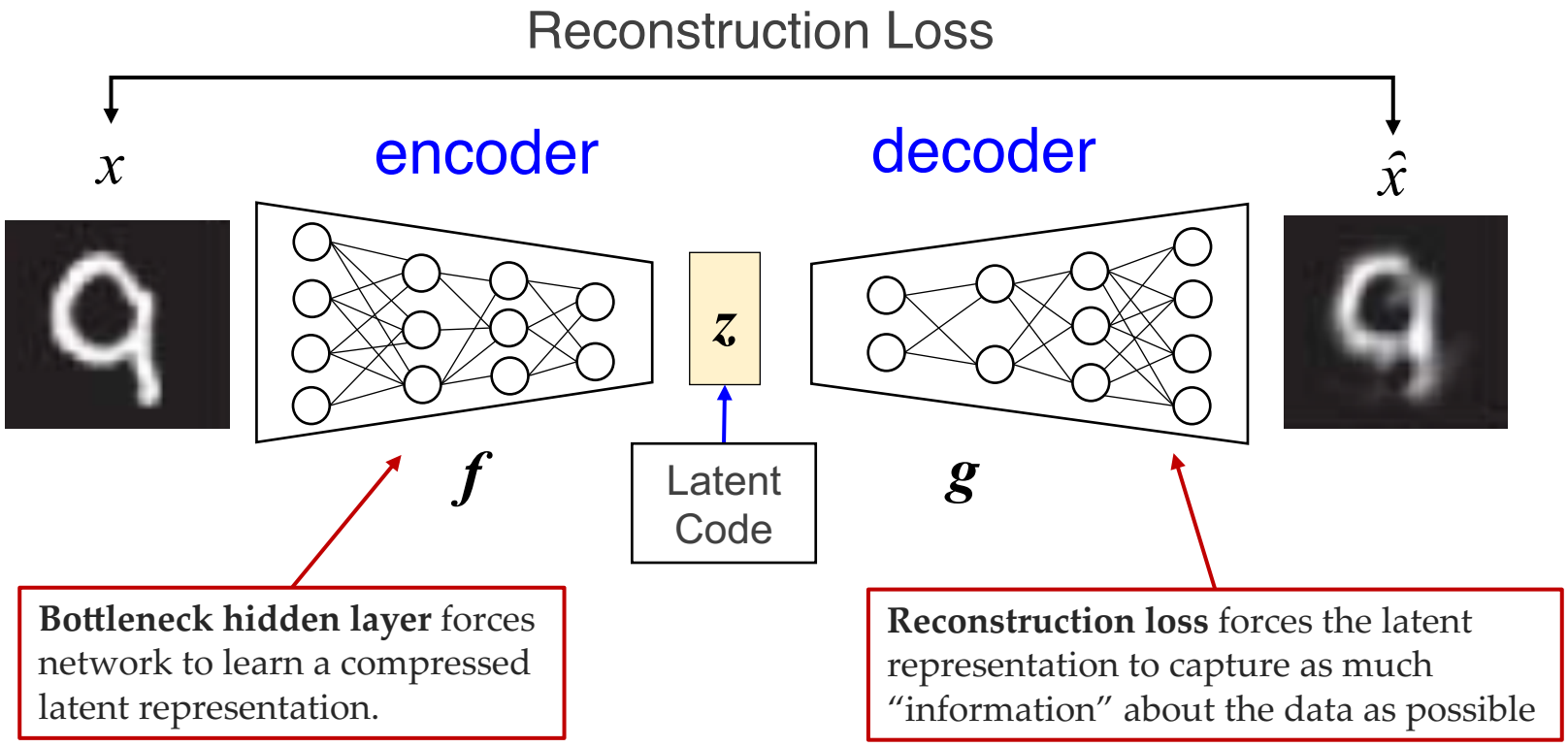
- **Auto-encoder**: an encoder-decoder **neural network** whose output tries to reconstruct the input.



Learns a **lower-dimensional** feature representation (z) from unlabeled training data (x).

Deep Auto-encoder

- Of course, the auto-encoder can be deep

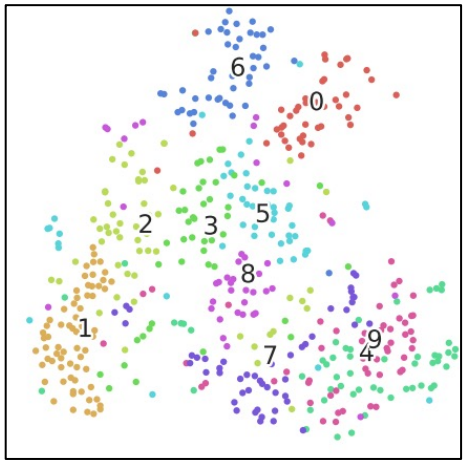


Hinton, Geoffrey E., and Ruslan R. Salakhutdinov. "Reducing the dimensionality of data with neural networks." *Science* 313.5786 (2006): 504-507

Auto-encoder vs PCA

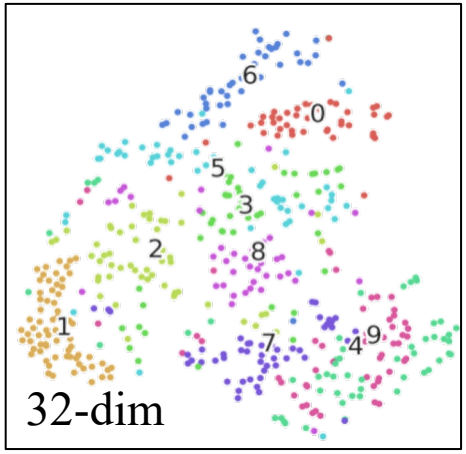
Example



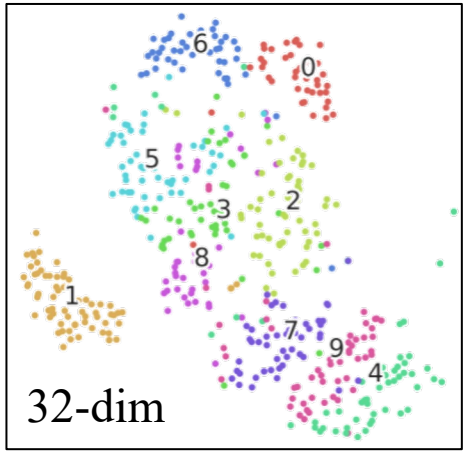


Pixels
(tSNE)

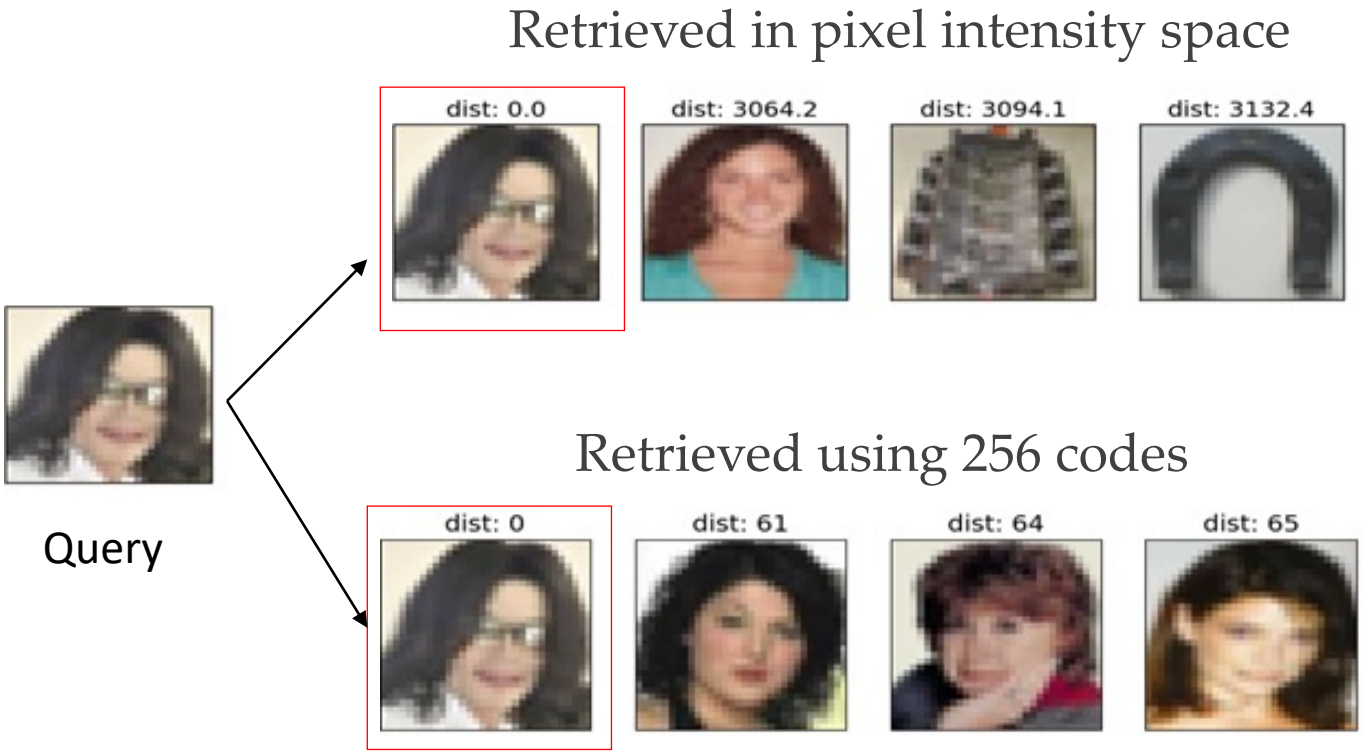
PCA



NN Auto-encoder

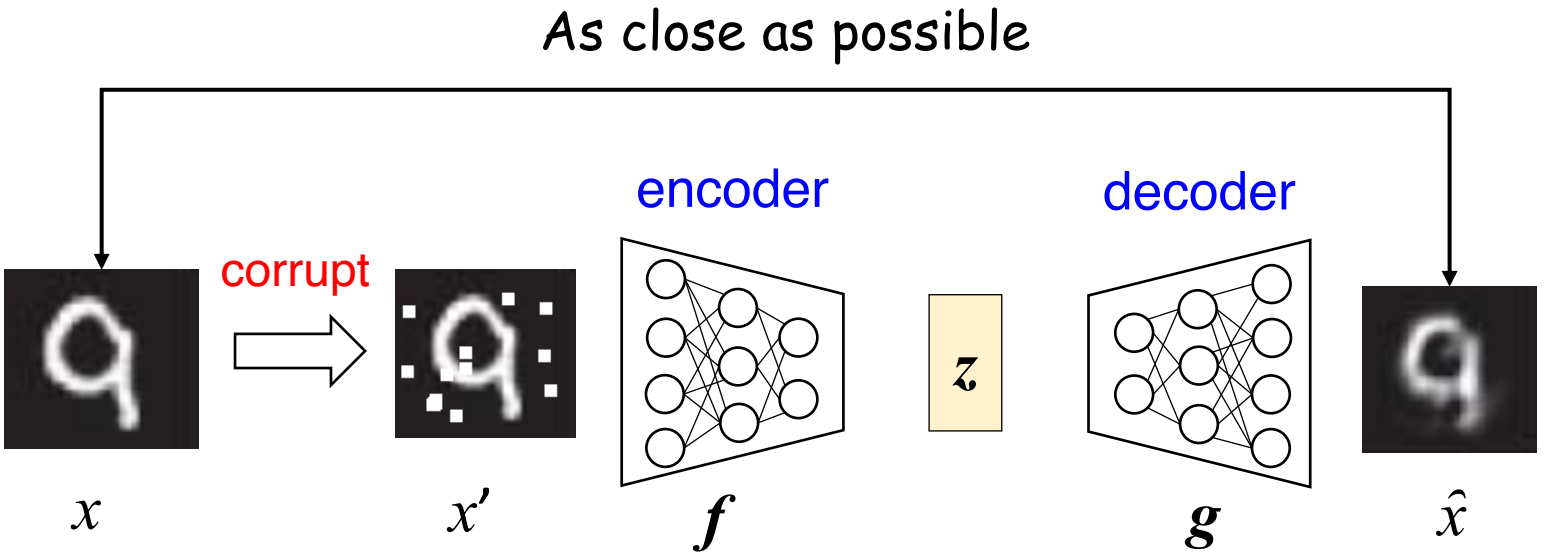


Applications – Image Search



Denoising Auto-encoder

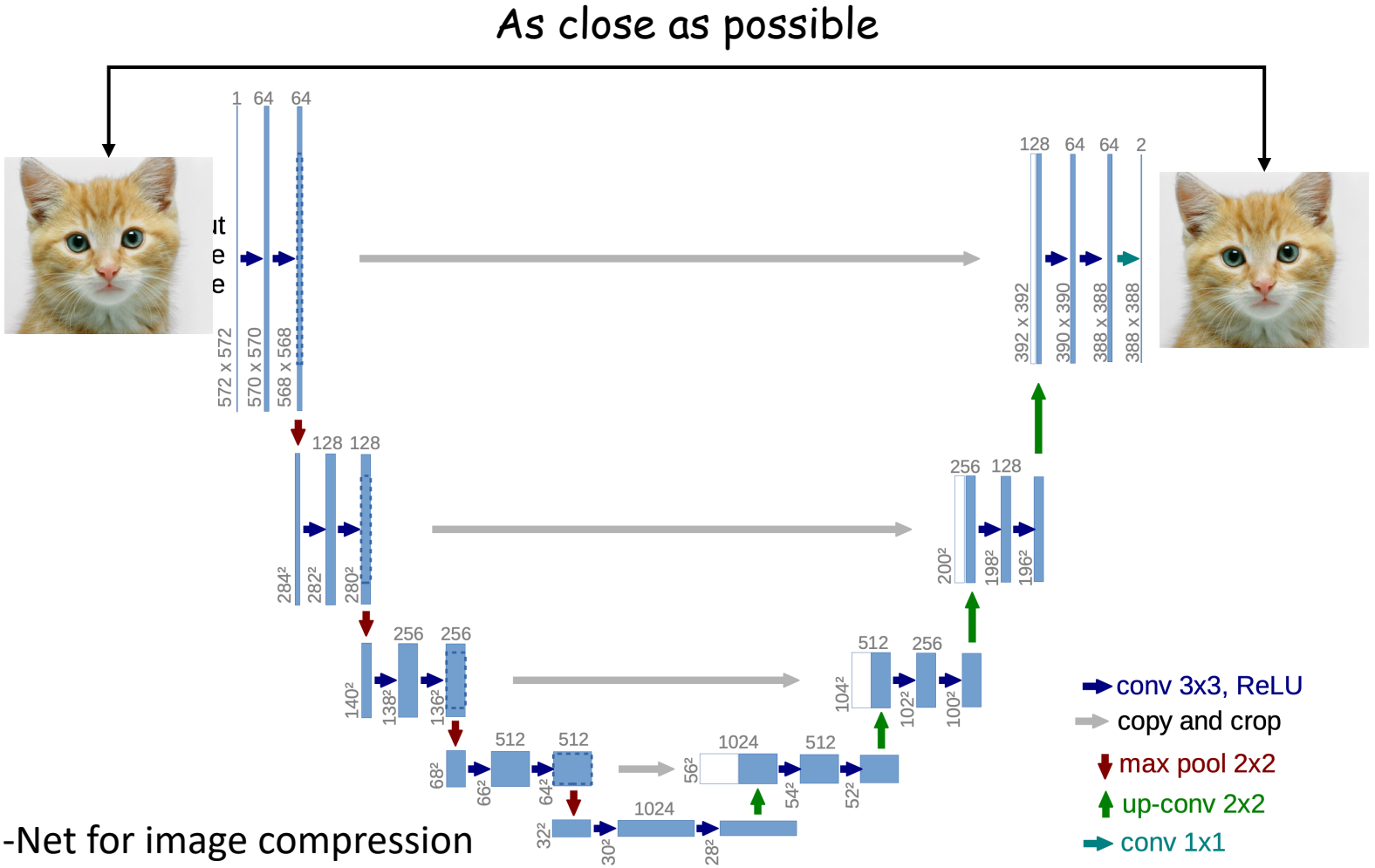
- An autoencoder that receives a **corrupted** data point as input and is trained to predict the original data point.



Applications: remove noise from data.

Vincent, Pascal, et al. "Extracting and composing robust features with denoising autoencoders." *ICML,* 2008.

Auto-encoder with CNN?



U-Net for image compression

What's Next?

Deep Generative Models



Learning to generate data

